МИНОБРНАУКИ РОССИИ

Воронежский государственный университет
Факультет прикладной математики, информатики и механики
Кафедра математического обеспечения ЭВМ

Цифровая обработка изображений

Лабораторная работа № 2

Выделение контуров и усредняющая фильтрация
Варианты 1 и 2

Выполнил(а): _____

Группа: _____

Преподаватель: Сырых А. С.

Воронеж, 2026

Постановка задачи

Необходимо реализовать на языке C++ обработку полутоновых изображений по двум выбранным вариантам задания.

Вариант 1 — выделение контуров с помощью операторов Лапласа, Робертса и Собела.

Вариант 2 — подавление гауссова шума пространственными усредняющими фильтрами. Зашумлённые изображения следует подготовить в графическом редакторе.

Перед выполнением обработки требуется завершить заданный класс Image: обеспечить чтение и сохранение PNG, доступ к пикселям, поддержку одного–четырёх каналов и независимое копирование изображений. Расширять интерфейс класса новыми методами не допускается. Работа должна сопровождаться тестовыми изображениями, которые показывают различия рассматриваемых методов.

Математическое описание алгоритма

Пусть f(x,y) задаёт яркость пикселя в позиции (x,y), а W и H — размеры изображения. Для восьмибитного представления допустимы значения от 0 до 255. Если исходное изображение цветное, перед обработкой вычисляется полутоновая яркость по формуле (1). Для одно- и двухканального входа используется существующий канал яркости.

$$f(x,y) = \mathrm{round}(0.299R + 0.587G + 0.114B). \quad (1)$$

При обращении за пределы растра берётся ближайший допустимый пиксель. Такое продолжение сохраняет постоянный фон у границ. Каждый метод читает исходный массив и формирует отдельное выходное изображение.

Алгоритмы выделения контуров

Граница объекта проявляется как быстрое изменение яркости. Для её обнаружения используются дискретные приближения производных: первые разности у Робертса и Собела и вторая разность у Лапласа.

Оператор Собела

В окне 3×3 рассчитываются изменения яркости по горизонтали и вертикали. Маски (2) объединяют разностную оценку с локальным сглаживанием в поперечном направлении:

$$S_x = [-1\ 0\ 1;\ -2\ 0\ 2;\ -1\ 0\ 1];$$

$$S_y = [-1\ -2\ -1;\ 0\ 0\ 0;\ 1\ 2\ 1]. \quad (2)$$

Компоненты градиента и его амплитуда определяются по (3). Суммирование ведётся по смещениям $i,j = -1,0,1$ относительно текущего пикселя:

$$G_x = \Sigma_i \Sigma_j S_x[j,i]\, f(x+i,y+j);$$

$$G_y = \Sigma_i \Sigma_j S_y[j,i]\, f(x+i,y+j);$$

$$M(x,y) = \sqrt{(G_x^2+G_y^2)}. \quad (3)$$

Для записи в восьмибитный канал результат ограничивается и округляется по (4). Эта операция не является растяжением яркостей по

максимуму конкретного изображения. Она также не выполняет пороговую бинаризацию:

$$Q(t) = \text{round}(\min(255, \max(0,t))); \; g(x,y) = Q(M(x,y)). \quad (4)$$

Алгоритм 1. Оператор Собела

1. Получить полутоновый вход по формуле (1).
2. Для каждого пикселя найти Gx и Gy с использованием масок (2).
3. Вычислить амплитуду по (3) и подготовить значение канала по (4).
4. Поместить значение в отдельное результирующее изображение.

Оператор Робертса

Вместо окна 3×3 используются две диагонали ячейки 2×2. Соответствующие маски записываются в виде (5):

$$Rx = [1\ 0;\ 0\ -1];\ Ry = [0\ 1;\ -1\ 0]. \quad (5)$$

Из этих масок следуют две разности (6). Амплитуда получается как корень из суммы их квадратов, после чего применяется ограничение (4):

$$a = f(x,y) - f(x+1,y+1);$$

$$b = f(x+1,y) - f(x,y+1);\ M(x,y) = \sqrt{(a^2+b^2)}. \quad (6)$$

Алгоритм 2. Оператор Робертса

1. Сформировать яркостное изображение по (1).
2. В каждом окне 2×2 вычислить диагональные разности по (6).
3. Найти амплитуду отклика и записать её с учётом (4).

Оператор Лапласа

В непрерывном представлении Лапласиан равен сумме вторых производных по двум пространственным координатам:

$$\Delta f = \partial^2 f/\partial x^2 + \partial^2 f/\partial y^2. \quad (7)$$

В работе применяется четырёхсвязное дискретное приближение с ядром (8):

$$K = [0\ 1\ 0;\ 1\ -4\ 1;\ 0\ 1\ 0]. \quad (8)$$

Поскольку вторая разность может иметь любой знак, для отображения используется её модуль. Формула расчёта приведена в (9):

$$L = f(x-1,y)+f(x+1,y)+f(x,y-1)+f(x,y+1)-4f(x,y);$$

$$g(x,y) = Q(|L|). \quad (9)$$

Алгоритм 3. Оператор Лапласа

1. Преобразовать вход в полутоновое представление по (1).
2. Вычислить вторую разность с использованием маски (8).
3. Взять модуль отклика и сформировать результат по (9).

Линейные усредняющие фильтры

Для уменьшения случайных колебаний яркости выходной пиксель вычисляется по его окрестности. При этом сглаживается не только шум, но и полезные изменения сигнала, поэтому выбор окна влияет на сохранность границ и мелких объектов.

Прямоугольный усредняющий фильтр (Box Blur)

Все значения квадратного окна имеют одинаковый вес. Для нечётного размера k радиус $r=(k-1)/2$, а число элементов равно k^2. Арифметическое усреднение описывается формулой (10):

$$g(x,y) = Q((1/k^2)\ \Sigma_i\ \Sigma_j\ f(x+i,y+j)),\ i,j = -r,\ldots,r. \quad (10)$$

Алгоритм 4. Прямоугольное усреднение

1. Выбрать нечётный размер окна и определить его радиус.
2. Присвоить каждому из k^2 элементов вес $1/k^2$.
3. Вычислить сумму значений окрестности с этими весами по (10).
4. Округлить результат и сохранить его в новом изображении.

Фильтр Гаусса (Gaussian Blur)

В отличие от равномерного окна, здесь центральные пиксели сильнее влияют на выход. Распределение весов задаётся гауссовой функцией (11), где σ определяет пространственный масштаб сглаживания:

$$G(i,j) = (1/(2\pi\sigma^2))\ \exp(-(i^2+j^2)/(2\sigma^2)). \quad (11)$$

На конечном окне веса необходимо нормировать по (12). Общий множитель $1/(2\pi\sigma^2)$ при делении сокращается, поэтому реализация вычисляет только экспоненциальную часть:

$$w(i,j) = G(i,j) / \Sigma_i\ \Sigma_j\ G(i,j);\ \Sigma_i\ \Sigma_j\ w(i,j) = 1. \quad (12)$$

После подготовки ядра выход определяется взвешенной суммой (13):

$$g(x,y) = Q(\Sigma_i \Sigma_j w(i,j) f(x+i,y+j)). \quad (13)$$

Алгоритм 5. Гауссово усреднение

1. Задать размер окна k и положительное значение σ.
2. Определить веса по (11) и нормировать их согласно (12).
3. Для каждого пикселя вычислить взвешенную сумму по (13).
4. Сформировать отдельный выходной массив с учётом (4).

Программное описание алгоритма

Программа написана на C++17. Объект Image отвечает за память, PNG и доступ к пикселям. Его исходный интерфейс сохранён; алгоритмы вынесены в пространство имён Filters. Данные хранятся в std::unique_ptr<uint8_t[]>, а копирование создаёт независимый массив. Для серого входа getColor возвращает яркость в r, устанавливая g и b в ноль; setColor сохраняет альфа-канал. При недопустимых координатах доступ безопасно игнорируется.

Функция Filters::toGrayscale формирует одно яркостное изображение по (1). Полученный объект используется при обработке, а исходные данные не изменяются. Вспомогательная sample повторяет ближайший граничный пиксель, put реализует ограничение и округление по (4).

Filters::applySobel принимает изображение по константной ссылке, получает его полутоновое представление и вызывает общий внутренний расчёт edges с выбором Собела. В нём выполняется алгоритм 1: накапливаются компоненты по маскам (2), затем вычисляется амплитуда (3). Соответствующие фрагменты кода приведены в приложении А.

Filters::applyRoberts использует тот же внутренний расчёт с другим режимом. Для каждой позиции берутся две диагональные разности по (6); их амплитуда преобразуется в допустимый диапазон. Это реализация алгоритма 2, представленная в приложении А.

Filters::applyLaplacian выбирает четырёхсвязную вторую разность и её модуль согласно (8), (9). Функция соответствует алгоритму 3; её реализация и общий расчёт edges также приведены в приложении А.

Filters::applyBoxBlur получает изображение и размер окна. Через внутреннюю функцию average она задаёт равные веса и выполняет алгоритм 4 по (10). Filters::applyGaussianBlur дополнительно проверяет положительность и конечность σ, формирует нормированные гауссовы веса и выполняет алгоритм 5 по (11)–(13). Эти фрагменты приведены в приложении Б.

Точка входа находится в Source.cpp. main выбирает вход и каталог результата, а runFilters последовательно запускает пять функций обработки: три контурные и две усредняющие. Арифметическое среднее применяется с окнами 3×3 и 5×5, Гаусс — с окном 5×5 при σ=1. Вместе с полутоновым входом записываются семь выходных PNG. Сохранение проверяет saveImage; стандартные исключения перехватываются в main, сопровождаются сообщением и кодом завершения 1.

Без аргументов используется готовый файл test_input/shapes_noisy.png. Можно передать два пути для своего изображения либо --random с необязательным seed для дополнительной случайной сцены. Она состоит из градиента, прямоугольников и тонких штрихов, имеет размер 320×224 и шум со стандартным отклонением 18. Если готовый пример отсутствует, такой тест создаётся автоматически. Этот режим не используется для численных таблиц отчёта: основные шумные входы подготовлены в ImageMagick согласно условию варианта 2.

В решении Visual Studio предусмотрены отдельные проекты Lab и Tests. Проверка содержит 18 тестовых случаев и 323 условия, все пройдены. Проверяются загрузка и сохранение, каналы, прозрачность, копирование, граничный доступ и заранее рассчитанные ответы фильтров. Например, на ступени 0→40 ответы Лапласа, Робертса и Собела равны соответственно 40, 57 и 160; среднее 3×3 для одиночного импульса 255 даёт 28.

Результат работы программы

Для эксперимента подготовлены два полутоновых изображения размером 256×192. В наборе А присутствуют крупные фигуры, ровный фон и плавное изменение яркости. В наборе Б размещены тонкие штрихи, клетки, окружность и диагональ. Первый набор показывает изменение шума на широких областях, второй позволяет обнаружить потерю небольших деталей. Для каждой сцены сохранены чистый эталон и шумная версия.

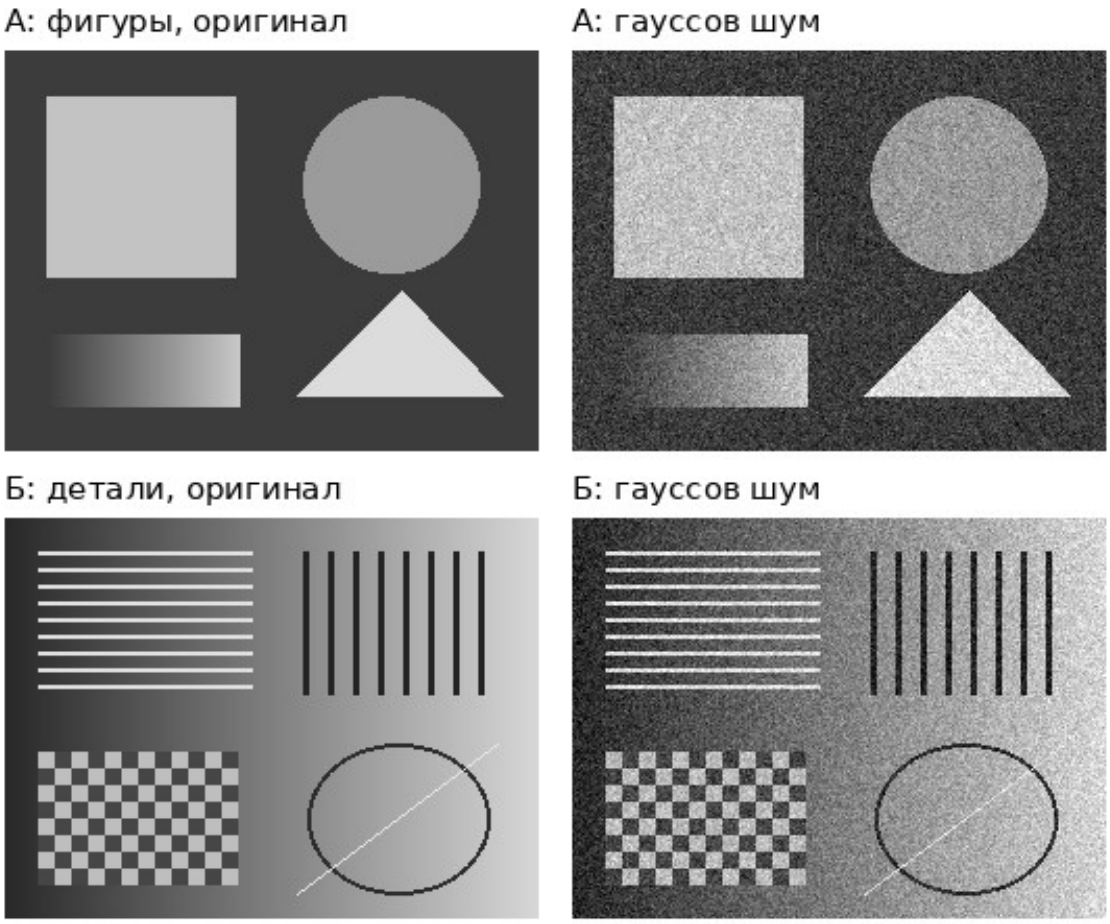


Рисунок 1. Слева — чистые изображения А и Б, справа — соответствующие версии с гауссовым шумом.

Шум добавлен вне C++-программы редактором ImageMagick с параметрами +noise Gaussian и -attenuate 0.9, с последующим сохранением в восьмибитный PNG. Начальные значения генератора равны 731 и 1201. Возможность применять другой графический редактор прямо предусмотрена вариантом 2.

Измеренное стандартное отклонение разности с чистым эталоном равно 17.92 для А и 17.92 для Б. Это характеристика шума в уровнях яркости; она не совпадает по смыслу с σ=1 пространственного гауссова фильтра.

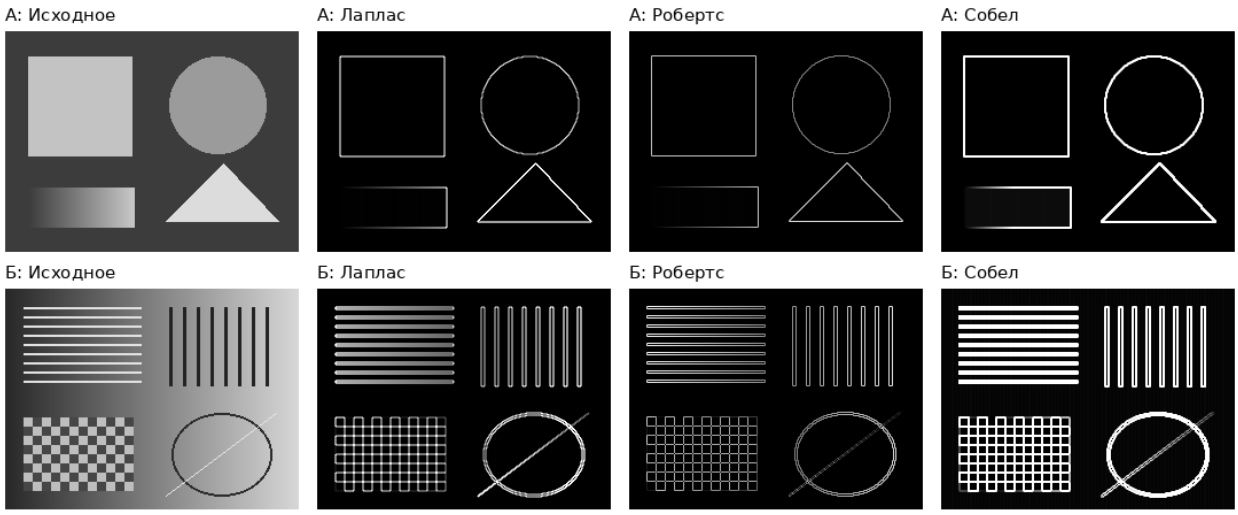


Рисунок 2. Контуры чистых сцен: сверху А, снизу Б. В каждом ряду — вход, Лаплас, Робертс и Собел.

Все три оператора реагируют на границы крупных фигур. У Лапласа после взятия модуля становятся видимыми отклики разных знаков с обеих сторон перепада, что объясняет двойную границу. Робертс даёт компактный отклик на диагональных разностях, а Собел формирует более широкий и яркий контур из-за маски 3×3 и её коэффициентов. Поэтому сравнение только по яркости линии недостаточно.

На сцене с деталями рядом расположенные границы штрихов и клеток могут сливаться. Внутри линейного градиента вторая разность близка к нулю, тогда как первые разности реагируют на плавное изменение яркости. Остаточные слабые отклики связаны в том числе с округлением значений при создании градиента.

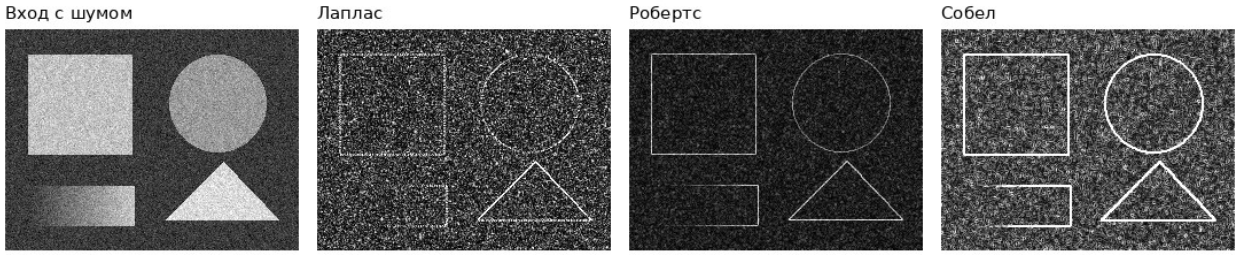


Рисунок 3. Шумный вход А и результаты контурных операторов в том же порядке.

При обработке шумного входа появляются ложные контуры на исходно однородных участках. Случайные отклонения также являются перепадами яркости и усиливаются разностными операциями. Поперечное сглаживание Собела снижает влияние отдельных колебаний, но не устраняет его полностью. Значения выше 255 насыщаются по (4), поэтому после ограничения яркость уже не отражает точную величину производной.

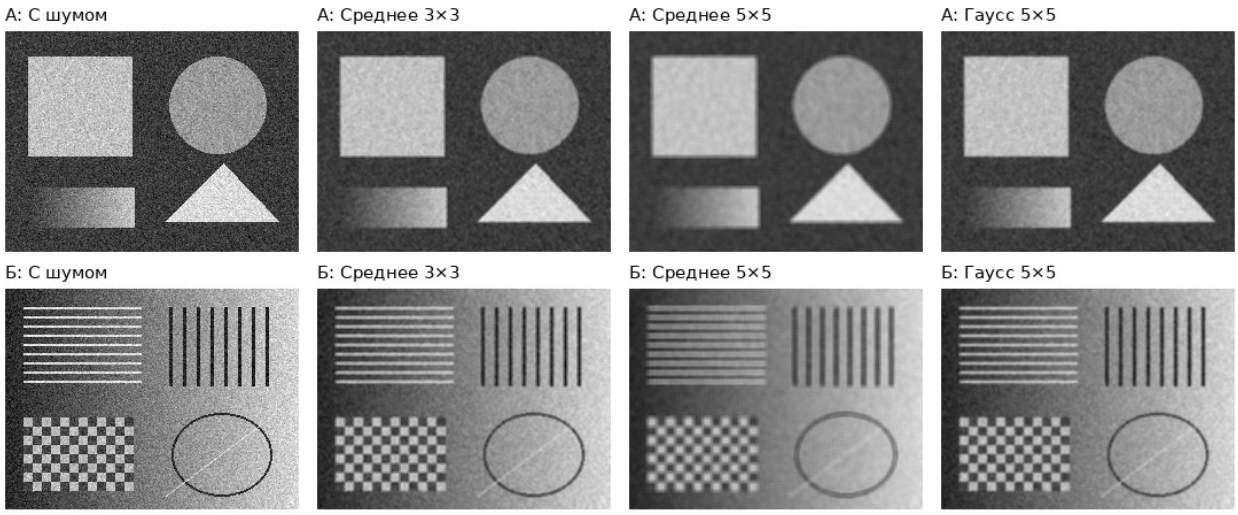


Рисунок 4. Подавление шума: вход, среднее 3×3, среднее 5×5 и Гаусс 5×5 с σ=1. Сверху А, снизу Б.

Для измерения ошибки результаты усреднения сопоставлены с чистыми эталонами. Используются показатели (14): меньшая MSE и большая PSNR означают более точное совпадение яркостей. Контурные карты так с исходным изображением не сравниваются, поскольку представляют другой вид сигнала.

$$MSE = (1/(WH))\ \Sigma_x \Sigma_y\ (g(x,y)-f(x,y))^2;$$

$$PSNR = 10\ \log_{10}(255^2/MSE),\ дБ. \quad (14)$$

Метод	A: MSE	A: PSNR, дБ	Б: MSE	Б: PSNR, дБ
Шумный вход	321.50	23.06	321.63	23.06
Среднее 3×3	111.66	27.65	437.16	21.72
Среднее 5×5	147.53	26.44	873.40	18.72
Гаусс 5×5	95.38	28.34	400.27	22.11

На крупных фигурах все усредняющие методы уменьшают ошибку относительно шумного входа. Наименьшую MSE среди проверенных настроек даёт Гаусс 5×5: 95.38 вместо 321.50 у входа. Небольшие веса удалённых соседей ограничивают смешивание разных областей по сравнению с равномерным окном 5×5.

Увеличение равномерного окна с 3×3 до 5×5 делает фон более гладким, но MSE на сцене А возрастает с 111.66 до 147.53. Причиной становится размытие границ. На сцене Б размеры тонких линий сопоставимы с окном, и все три настройки повышают общую MSE относительно шумного входа. Снижение зернистости не компенсирует потерю полезных элементов.

Следовательно, результат необходимо оценивать одновременно по ровным участкам и деталям. Из трёх исследованных настроек гауссово окно 5×5 при $\sigma=1$ имеет меньшую ошибку на обеих сценах, однако это наблюдение не доказывает его превосходство при произвольных изображениях и параметрах шума.

Заключение

В работе выполнены два выбранных варианта: выделение контуров и пространственное подавление гауссова шума усреднением. Реализован заданный класс Image, а функции обработки отделены от хранения изображения. Четыре демонстрационных запуска сформировали 28 выходных PNG, проверка из 18 тестовых случаев завершилась успешно.

Операторы Лапласа, Робертса и Собела выявляют перепады яркости, но различаются формой отклика и реакцией на шум. Усредняющие фильтры ослабляют колебания фона ценой размытия. Большое окно не гарантирует меньшую ошибку: сохранность мелких элементов зависит от масштаба окрестности. Иллюстрации и численные оценки подтверждают эти свойства и согласуются с приведёнными формулами.

Приложение А

Фрагменты Filters.cpp для выделения контуров. Сначала приведены общие вспомогательные функции и преобразование яркости, затем расчёт трёх операторов. Общие функции также используются листингом приложения Б. Полный проект содержит необходимые объявления и точку входа.

```
#include "Filters.h"
#include <algorithm>
#include <cmath>
#include <vector>
namespace {
enum class Edge { Laplace, Roberts, Sobel };
void requireGray(const Image& image) {
    if (image.width() <= 0 || image.height() <= 0 || image.nChannels()
!= 1)
           throw std::invalid_argument("Expected a nonempty grayscale
image");
}
void requireWindow(int window) {
    if (window < 1 || window > 31 || window % 2 == 0)
         throw std::invalid_argument("Window must be odd and between 1
and 31");
}
int sample(const Image& image, int x, int y) {
    // Replicate the closest border pixel; do not add artificial black
edges.
    x = std::clamp(x, 0, image.width() - 1);
    y = std::clamp(y, 0, image.height() - 1);
    int r = 0, g = 0, b = 0;
    image.getColor(x, y, r, g, b);
    return r;
}
void put(Image& image, int x, int y, double value) {
     const int v = static_cast<int>(std::lround(std::clamp(value, 0.0,
255.0)));
    image.setColor(x, y, v, v, v);
}
}
namespace Filters {
Image toGrayscale(const Image& input) {
    if (input.width() <= 0 || input.height() <= 0)
        throw std::invalid_argument("Empty input image");
    Image result(input.width(), input.height(), 1);
    for (int y = 0; y < input.height(); ++y)
        for (int x = 0; x < input.width(); ++x) {
            int r = 0, g = 0, b = 0;
            input.getColor(x, y, r, g, b);
```

```
            put(result, x, y, input.nChannels() < 3 ? r : 0.299*r +
0.587*g + 0.114*b);
        }
    return result;
}
static Image edges(const Image& gray, Edge method) {
    requireGray(gray);
    Image result(gray.width(), gray.height(), 1);
    const int sx[3][3] = {{-1,0,1}, {-2,0,2}, {-1,0,1}};
    const int sy[3][3] = {{-1,-2,-1}, {0,0,0}, {1,2,1}};
    for (int y = 0; y < gray.height(); ++y)
        for (int x = 0; x < gray.width(); ++x) {
            double value = 0;
            if (method == Edge::Laplace) {
                                value  =  std::abs(sample(gray,x-1,y)  +
sample(gray,x+1,y) +
                                                        sample(gray,x,y-1)  +
sample(gray,x,y+1) - 4*sample(gray,x,y));
            } else if (method == Edge::Roberts) {
                const int a = sample(gray,x,y) - sample(gray,x+1,y+1);
                const int b = sample(gray,x+1,y) - sample(gray,x,y+1);
                value = std::hypot(a, b);
            } else {
                int gx = 0, gy = 0;
                for (int j = -1; j <= 1; ++j)
                    for (int i = -1; i <= 1; ++i) {
                        const int p = sample(gray,x+i,y+j);
                        gx += p * sx[j+1][i+1];
                        gy += p * sy[j+1][i+1];
                    }
                value = std::hypot(gx, gy);
            }
            put(result, x, y, value);
        }
    return result;
}
Image applySobel(const Image& source) {
    return edges(toGrayscale(source), Edge::Sobel);
}
Image applyRoberts(const Image& source) {
    return edges(toGrayscale(source), Edge::Roberts);
}
Image applyLaplacian(const Image& source) {
    return edges(toGrayscale(source), Edge::Laplace);
}
}
```

Приложение Б

Фрагменты Filters.cpp для усредняющих фильтров. Внутренний расчёт average используется функциями applyBoxBlur и applyGaussianBlur; toGrayscale, sample и put приведены в приложении А.

```
namespace Filters {
static Image average(const Image& gray, int window, double sigma) {
    requireGray(gray);
    requireWindow(window);
    if (!std::isfinite(sigma) || sigma < 0)
              throw  std::invalid_argument("Sigma  must  be  finite  and
nonnegative");
    const int radius = window / 2;
    std::vector<double> weights(window * window, 1.0);
    double total = 0;
    for (int j = -radius; j <= radius; ++j)
        for (int i = -radius; i <= radius; ++i) {
            double& w = weights[(j+radius)*window+i+radius];
            if (sigma > 0) w = std::exp(-(i*i+j*j)/(2*sigma*sigma));
            total += w;
        }
    for (double& w : weights) w /= total;
    Image result(gray.width(), gray.height(), 1);
    for (int y = 0; y < gray.height(); ++y)
        for (int x = 0; x < gray.width(); ++x) {
            double sum = 0;
            for (int j = -radius; j <= radius; ++j)
                for (int i = -radius; i <= radius; ++i)
                            sum += weights[(j+radius)*window+i+radius] *
sample(gray,x+i,y+j);
            put(result,x,y,sum);
        }
    return result;
}
Image applyBoxBlur(const Image& source, int kernelSize) {
    return average(toGrayscale(source), kernelSize, 0.0);
}
Image  applyGaussianBlur(const  Image&  source,  int  kernelSize,  double
sigma) {
    if (!std::isfinite(sigma) || sigma <= 0)
         throw std::invalid_argument("Gaussian sigma must be finite and
positive");
    return average(toGrayscale(source), kernelSize, sigma);
}
}
```